

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



≡ LangChain overview

LangChain overview

📄 Copy page



Agent = Model + Harness. LangChain provides `create_agent` : a minimal, highly configurable harness. The harness is everything around the model loop: the prompt, the tools, and any middleware that shapes behavior. Start with the primitives and compose exactly what your use case needs. Supports [OpenAI](#), [Anthropic](#), [Google](#), and [more](#).



LangChain vs. LangGraph vs. Deep Agents

Start with [Deep Agents](#) for a “batteries-included” agent with features like automatic context compression, a virtual filesystem, and subagent-spawning. Deep Agents are built on [LangChain agents](#) which you can also use directly.

Use [LangChain](#) (`create_agent`) for a highly customizable harness, easily tailored to your use case and data.

Use [LangGraph](#), our low-level orchestration framework, for advanced needs combining deterministic and agentic workflows.

Use [LangSmith](#) to trace, debug, and evaluate agents built with any of these frameworks. Follow the [tracing quickstart](#) to get set up. We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

Create an agent

This example demonstrates how to create a simple LangChain agent with a custom tool:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

```
return f"It's always sunny in {city}!"

agent = create_agent(
    model="openai:gpt-5.5",
    tools=[get_weather],
    system_prompt="You are a helpful assistant",
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}]}
)
print(result["messages"][-1].content_blocks)
```

See the [Installation instructions](#) and [Quickstart guide](#) to get started building your own agents and applications with LangChain.



Use [LangSmith](#) to trace requests, debug agent behavior, and evaluate outputs. Set `LANGSMITH_TRACING=true` and your API key to get started.

Core benefits

Standard model interface

Use one interface for chat models, embeddings, and more across providers. Switch models with minimal code changes and keep your application portable as requirements evolve.

Highly configurable harness

Start with `create_agent` as a minimal harness and add capabilities incrementally through middleware. Compose only what your use case needs, from guardrails and retries to routing and custom tool policies.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

LangChain Docs

of LangGraph. This allows us to take advantage of LangGraph's durable execution, human-in-the-loop support, persistence, and more.

[Learn more >](#)

transitions, and latency in one place. Find failure modes, evaluate quality, and improve agent behavior with execution data.

[Learn more >](#)

 [Connect these docs](#) to Claude, VSCode, and more via MCP for real-time answers.

 [Edit this page on GitHub](#) or [file an issue](#).

Was this page helpful?

 Yes

 No

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



[LangChain Academy](#)[Careers](#)[Contact Sales](#)[Blog](#)